

CHUYÊN ĐỀ 2: BÀI TẬP 2

PHÂN TÍCH THỜI GIAN THỰC THI CHƯƠNG TRÌNH

Lê Văn Minh Trí 106220238

Đinh Hoàng Thuận 106220271

BT2: Phân tích thời gian thực thi chương trình

1. Viết 2 thuật toán để sắp xếp 1 mảng gồm N phần tử ($N \leq 10^5$):

a. Dùng giải thuật sắp xếp chèn/nổi bọt.

b. Dùng giải thuật sắp xếp sử dụng cây heap.

2. Báo cáo thời gian chạy thực (running time) của 2 thuật toán trên khi N tăng dần. Nếu thời gian chạy > 5 s thì xem như “time out” và không cần báo cáo thêm.

3. Kết luận về khả năng chạy của 2 thuật toán (N lớn nhất mà thuật toán có thể xử lý được với thời gian thực thi không quá 5 s).

4. Phát triển/đánh giá giải thuật trên các điều kiện môi trường, cấu hình máy tính hoặc ngôn ngữ khác nhau (tùy chọn).

Lời giải BT 1, 2, 3

Lời giải bằng ngôn ngữ Python: tệp *BT2/Python-Solution.ipynb*

Lời giải bằng ngôn ngữ C++: tệp *BT2/C++-Solution.ipynb*

Hai tệp trên chứa phân cài đặt thuật toán, đo thời gian chạy và kiểm tra kết quả sắp xếp. Bản C++ được thiết kế để chạy trực tiếp trên Google Colab bằng g++.

2. Báo cáo thời gian chạy thực

Theo kết quả của chương trình Python, khi N tăng dần, thời gian chạy của hai thuật toán có sự thay đổi rõ rệt. Trong lần chạy tham chiếu của notebook, Heap Sort vượt ngưỡng 5 s ở khoảng $N = 3500$, trong khi Insert Sort tại $N = 3500$ vẫn chưa đạt 1 s.

Nhận xét: Trong phiên bản Python được cung cấp, Heap Sort được triển khai bằng cơ chế max heap kết hợp đệ quy, đồng thời có thao tác chèn/xóa đầu danh sách. Vì vậy thời gian chạy thực tế của phiên bản này lớn hơn Insert Sort trong vùng N được khảo sát, mặc dù xét về lý thuyết Heap Sort có độ phức tạp tốt hơn.

Ngôn ngữ	Thuật toán	Kết quả tham chiếu
Python	Heap Sort	Vượt 5 s khoảng $N = 3500$; N lớn nhất ghi nhận: 3437
Python	Insert Sort	N lớn nhất ghi nhận: 10395 (phụ thuộc lần chạy/random list)

3. Kết luận về khả năng chạy

Với điều kiện thời gian thực thi không quá 5 s, kết quả tham chiếu của chương trình Python cho thấy:

- Heap Sort: N lớn nhất khoảng 3437.
- Insert Sort: N lớn nhất khoảng 10395 trong lần chạy được ghi nhận. Giá trị này mang tính tương đối vì danh sách ngẫu nhiên có thể khác giữa các lần chạy.

Kết luận: Trong phiên bản Python hiện tại, Insert Sort có thời gian chạy thực tế tốt hơn Heap Sort ở khoảng N được khảo sát. Nguyên nhân chủ yếu là cách triển khai Heap Sort trong notebook sử dụng đệ quy và các thao tác trên list có chi phí lớn.

Kết quả thực nghiệm bằng C++

Phiên bản C++ sử dụng Insert Sort $O(N^2)$ và Heap Sort chuẩn $O(N \log N)$, được biên dịch với g++ - O2. Kết quả benchmark thu được trên môi trường chạy hiện tại như sau:

N	Insert Sort (s)	Heap Sort (s)
100	0.000004	0.000007
500	0.000037	0.000040
1000	0.000136	0.000087
2000	0.000505	0.000194
5000	0.003229	0.000497
10000	0.013097	0.001070
20000	0.052424	0.002461
50000	0.316167	0.006380
100000	1.279838	0.014729

Nhận xét C++: Khi N còn nhỏ, thời gian của hai thuật toán khá gần nhau. Khi N tăng, Heap Sort nhanh hơn rõ rệt. Ở $N = 100000$, Insert Sort mất khoảng 1.28 s trong khi Heap Sort chỉ khoảng 0.0147 s. Cả hai vẫn nằm dưới giới hạn 5 s ở mức $N = 100000$ trong lần benchmark này.

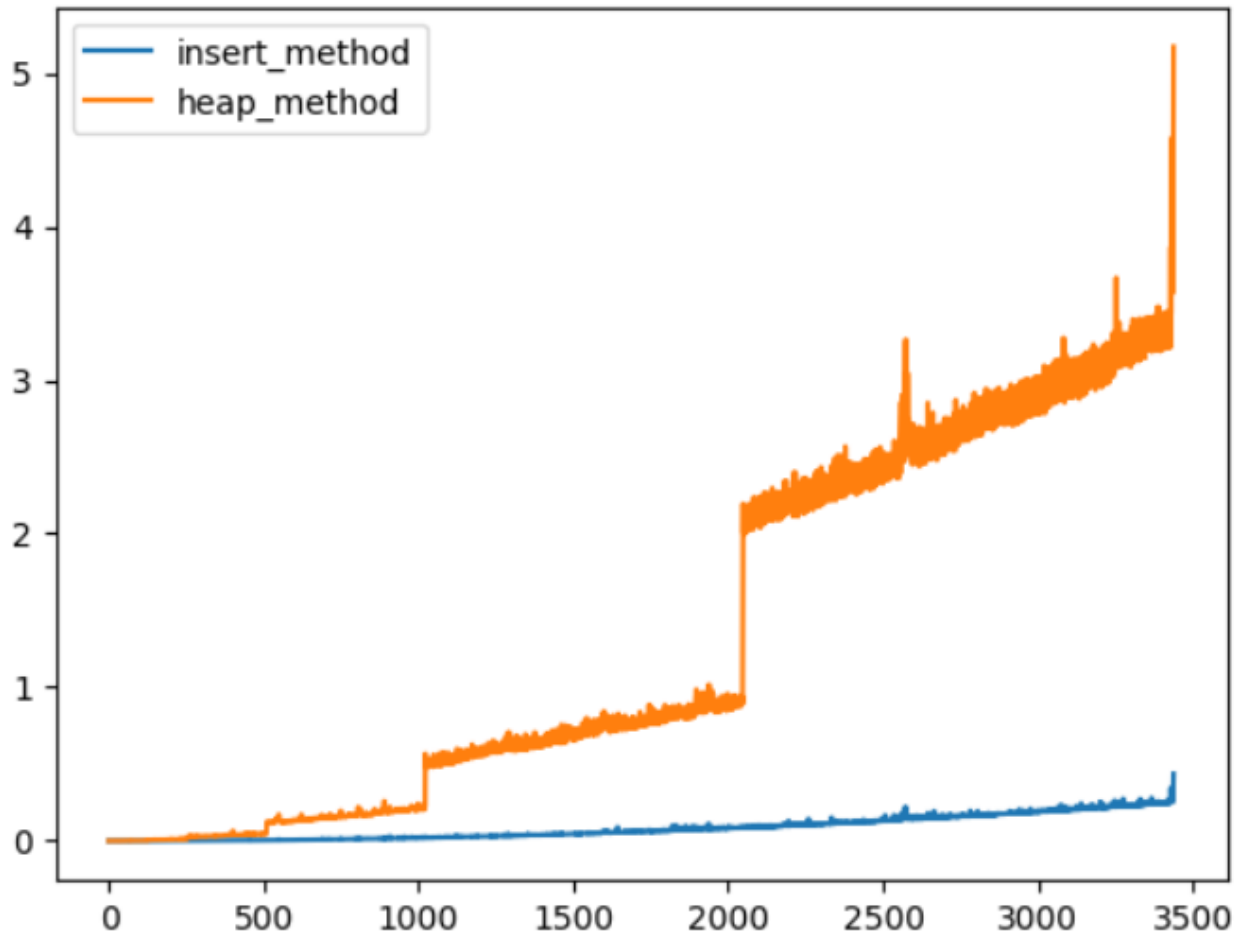
4. Đánh giá thời gian chạy với các ngôn ngữ khác nhau

Khi so sánh kết quả giữa Python và C++, có thể thấy môi trường và cách triển khai ảnh hưởng đáng kể đến running time. Trong notebook Python, phiên bản Heap Sort sử dụng đệ quy và thao tác trên list nên thời gian chạy tăng nhanh. Trong khi đó, phiên bản C++ sử dụng Heap Sort chuẩn với heapify dạng lặp, giúp giảm đáng kể chi phí thực thi.

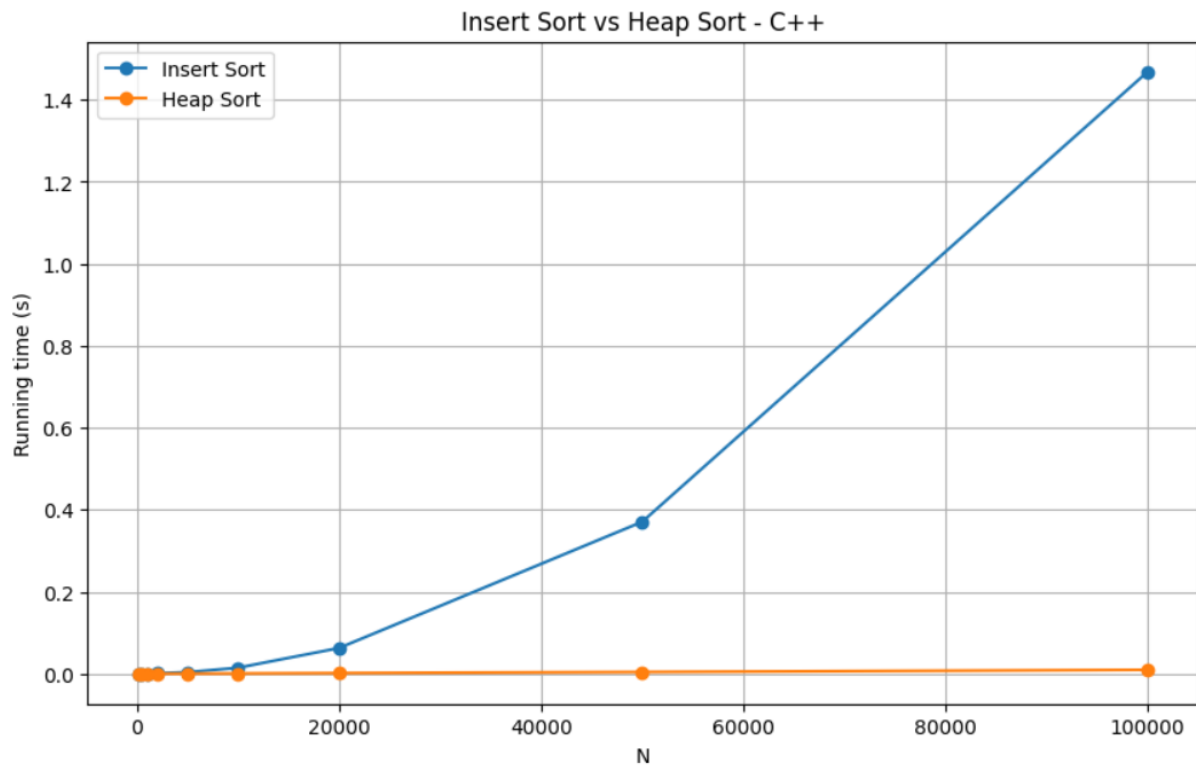
Đối với Python: Heap Sort vượt ngưỡng 5 s khoảng $N = 3500$; Insert Sort có thể xử lý khoảng $N = 10395$ trong lần chạy tham chiếu.

Đối với C++: Trong benchmark hiện tại, tại $N = 100000$, Insert Sort khoảng 1.28 s và Heap Sort khoảng 0.0147 s, đều chưa vượt giới hạn 5 s.

Hình 1. Biểu đồ thời gian chạy thực của Insert Sort và Heap Sort bằng Python.



Hình 2. Biểu đồ thời gian chạy thực của Insert Sort và Heap Sort bằng C++.



=> **Kết luận chung:** C++ cho thời gian thực thi nhanh hơn rõ rệt trong benchmark này. Đặc biệt, Heap Sort C++ có lợi thế lớn khi N tăng do sử dụng thuật toán $O(N \log N)$ và cách cài đặt hiệu quả.